

NEURONAV: GAZE EYE TRACKING & NEURAL INTERFACE

Technical Architecture, Systems Deep Dive & Comprehensive Interview Handbook

Candidate / Author: Interview Preparation	Primary Stack: Python, OpenCV, MediaPipe, React 19, Flask
Domain: Computer Vision, BCI, Multimodal HCI, NLP	Target Roles: Software Engineer, Full Stack, Computer Vision, AI/ML

1. Executive Summary & Elevator Pitch

Use these scripted responses to introduce the project concisely in interviews:

■■ **30-Second Elevator Pitch:**
'NeuroNav is a multimodal assistive computing platform enabling hands-free system navigation and mental wellness telemetry. It leverages real-time computer vision (MediaPipe Face Landmarker Tasks API) to map ocular gaze and distinct blink patterns to OS-level mouse movement and clicks. It interfaces with a React 19 single-page dashboard offering voice-activated control (Web Speech API), real-time Arduino EEG telemetry (Web Serial API at 230,400 baud), and BERT-based conversational sentiment analytics.'

2. Project Directory Structure & Architecture Breakdown

The codebase is logically split into an independent backend computer vision server and a modern React frontend client application:

File / Directory	Layer	Core Technical Responsibility
backend/eye_tracking_mouse_control.py	Python / CV	OpenCV camera capture (30 FPS @ 720p), MediaPipe Face Landmarker (478 3D points), Exponential Moving Average (EMA) coordinate mapping, temporal blink state machine, and PyAutoGUI cursor actuation.
backend/face_landmarker.task	Model Asset	Pre-trained MediaPipe Tasks vision bundle enabling real-time on-device landmark inference without external cloud APIs.
backend/flask_server.py	Python / Flask	REST API on port 5001. Asynchronously orchestrates CV script lifecycle via subprocess.Popen and serves conversational sentiment data.
frontend/src/App.js	React SPA	Central client-side routing (React Router Dom) managing Login, Setup/Calibration, Dashboard, and Gaming routes.
frontend/src/components/LoginPage.js	React Component	Hands-free authentication interface integrating Web Speech API (react-speech-recognition) for continuous vocal wake words ('hello', 'login', 'bye').
frontend/src/components/SetupPage.js	React Component	Interactive 3-step hardware initialization and calibration wizard communicating with Flask backend via Axios REST endpoints.

frontend/src/components/NeuroNavDashboard.js	React Component	Central neural wellness cockpit with Recharts time-series data visualizations, cognitive health status categorizers, and system metrics.
frontend/src/components/EEGWave.js	React / BCI	Web Serial API (230,400 baud) stream reader, real-time HTML5 2D Canvas EEG waveform renderer, and hardware multi-blink action controller.
frontend/src/components/Sentimentanalysis.py	NLP / PyTorch	BERT sequence classification pipeline evaluating emotional indices (Sentiment, Anxiety, Depression, Stress).
start.sh	Bash Script	Root process orchestration script launching Flask and React simultaneously with POSIX signal trapping (SIGINT/SIGTERM).

3. Deep Dive into Core Technical Modules

A. Real-Time Gaze Tracking & Anti-Jitter Filter

- **De-coupled Asymmetric Tracking:** The right eye iris center (Landmark 473) controls cursor position, while the left eye vertical eyelid aperture (Landmarks 159 & 145) handles click detection. This decouples navigation from clicking, solving the notorious *Midas Touch* problem.
- **Dynamic Screen Calibration:** Captures 200 initial frames to calculate bounding range [x_min, x_max, y_min, y_max] with a 5% safety margin. Clamps and normalizes incoming landmark points to a [0, 1] range before mapping via linear interpolation (np.interp).
- **Exponential Moving Average (EMA) Smoothing:** Dampens micro-saccades and camera noise without introducing sluggish latency:
$$\text{screen_x} = 0.6 * \text{prev_screen_x} + 0.4 * \text{raw_screen_x}$$
- **Temporal Blink State Machine:** Calculates eyelid distance $|y_{\text{top}} - y_{\text{bottom}}| < 0.01$. Only eye closures between 50ms and 500ms trigger a click, rejecting involuntary twitches and prolonged resting eyes. A 700ms cooldown prevents double-click bouncing.

B. Subprocess Lifecycle Management in Flask

- Heavy computer vision loops cannot run inside Flask's request threads without locking the server. The backend spawns `eye_tracking_mouse_control.py` as an isolated OS process via `subprocess.Popen([sys.executable, script_path])`.
- Implements **fail-fast detection**: after spawning, the server polls the PID after 800ms; if it exited prematurely (e.g. camera permission denied), it captures `stderr` and returns an informative HTTP 500 JSON payload.

C. Direct Browser-to-Hardware BCI (Web Serial API & Canvas)

- **Web Serial API:** Uses `navigator.serial.requestPort()` with a 230,400 baud rate to establish a direct, driverless serial stream between Arduino/EEG hardware and the client browser.
- **Non-Blocking Canvas Rendering:** Serial packets are buffered in a streaming text decoder. Waveforms are drawn directly on an HTML5 2D `<canvas>` to achieve high refresh rates without triggering React DOM re-renders.
- **Multi-Blink Gesture Macros:** High-voltage bioelectric spikes ($>700\mu\text{V}$) counted within a 3-second sliding window trigger environmental modes: 2 blinks = brightness slider, 4 blinks = Focus Mode (100% brightness + audio), 8 blinks = Relax Mode (30% brightness + audio).

4. Key Engineering Challenges & Solutions

Challenge	Root Cause / Difficulty	Architectural Solution
Gaze Jitter vs. Input Lag	Human eyes constantly produce involuntary micro-saccades. Heavy averaging causes cursor drag.	Employed calibrated min-max coordinate clamping coupled with a tuned 0.6 Exponential Moving Average (EMA) filter.
The 'Midas Touch' Problem	Using the same eye for moving and clicking triggers false clicks whenever the user blinks naturally.	Separated ocular responsibilities (Right eye = position, Left eye = click) and enforced temporal windowing (50ms - 500ms) with 700ms cooldown.
Cross-Process Orchestration	Managing long-running Python vision processes from a web dashboard without zombie processes.	Managed background subprocesses in Flask with explicit PID tracking, graceful termination, and POSIX signal trapping in start.sh.
High-Frequency Serial Data in React	230.4k baud serial streaming causes React component state updates to bottleneck the UI thread.	Decoupled telemetry state from React virtual DOM by streaming chunks to a ref buffer and rendering on an HTML5 2D Canvas.

5. Top 10 Technical Interview Questions & Answers

Q1: Why choose MediaPipe Face Landmarker over dlib or OpenCV Haar Cascades?

Answer: MediaPipe Face Landmarker provides 478 3D landmarks (including dedicated sub-pixel iris points like #473) using lightweight, hardware-accelerated deep neural networks capable of 30+ FPS on CPU. Haar cascades only detect 2D bounding boxes, and dlib's 68-point model is computationally heavier and lacks fine-grained iris landmarks.

Q2: How does your coordinate mapping algorithm work mathematically?

Answer: During calibration, the system records 200 frames of eye movement to establish $[x_{min}, x_{max}, y_{min}, y_{max}]$. During runtime, normalized coordinates $(x - x_{min}) / (x_{max} - x_{min})$ are scaled via linear interpolation (`np.interp`) to screen dimensions (e.g. 1920x1080) and smoothed using an Exponential Moving Average: $S_t = \alpha * S_{t-1} + (1-\alpha) * Y_t$ where $\alpha = 0.6$.

Q3: How did you solve the Midas Touch problem?

Answer: By decoupling tracking into an asymmetric multi-facial scheme: the right eye coordinates navigation, while the left eye vertical aperture handles clicks. Blinks are validated against a strict temporal window (50ms to 500ms) with a 700ms debounce cooldown.

Q4: Why use the Web Serial API instead of WebSockets for Arduino/EEG communication?

Answer: The Web Serial API enables direct, zero-latency, driverless peer-to-peer communication between the browser and USB microcontrollers. It removes the need for users to run a local daemon/WebSocket relay, dramatically simplifying deployment.

Q5: How do you handle process termination and prevent zombie processes?

Answer: Flask maintains a global reference to the spawned subprocess.Popen instance. The `/stop-eye-tracking` endpoint checks `poll()` and executes `terminate()`. Additionally, the root `start.sh` script registers a POSIX trap on `SIGINT/SIGTERM` to kill all child PIDs upon exit.

Q6: How does head movement affect gaze accuracy, and how would you improve it?

Answer: Currently, coordinates are frame-relative. To make tracking invariant to head rotation, we can implement 3D Pose Estimation: using solvePnP on rigid facial landmarks (nose bridge, chin) to compute head rotation vectors and subtracting head yaw/pitch from raw gaze vectors.

Q7: How is conversational sentiment analyzed in this system?

Answer: The NLP engine utilizes a pre-trained BERT sequence classification model (bert-base-uncased) in PyTorch. It calculates a weighted softmax distribution across 5 sentiment classes and derives anxiety, depression, and stress indices plotted over time with Recharts.

Q8: Why did you choose React 19 for the frontend architecture?

Answer: React provides component-driven isolation for managing asynchronous states (Web Speech API, Web Serial streams, REST polling, and dynamic SVG chart re-renders). Pairing it with Framer Motion ensures smooth 60 FPS transitions.

Q9: What security considerations exist in this architecture?

Answer: Camera access and serial port access are protected by browser security sandboxes (requiring explicit user permissions). For production, Flask endpoints should require JWT authentication and CORS should be restricted to trusted origins.

Q10: What would you do differently in a Production V2?

Answer: 1) Port the eye-tracking model entirely into the browser via MediaPipe WebAssembly / ONNX Web to eliminate Python backend dependencies. 2) Replace EMA with a dynamic One Euro (1€) filter. 3) Store user calibration profiles in a cloud database.

6. Future Enhancements & Production Roadmap

- **Full Browser WebAssembly Migration:** Run MediaPipe Face Landmarker client-side in WebAssembly/WebGL using the JavaScript Tasks API to remove Python/OpenCV desktop requirements.
- **1€ Filter Implementation:** Dynamically adapt smoothing coefficients based on eye velocity—high smoothing during fixations, zero latency during rapid saccades.
- **Cloud Profiles & Telemetry:** Store user calibration matrices, preferred sensitivities, and longitudinal wellness data in PostgreSQL with OAuth2 authentication.